CS 2143 · OBJECT-ORIENTED PROGRAMMING

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

GRADING STRIP

```
1-C*  2-D   3-B   4-A   5-D*  6-C   7-A   8-B   9-D   10-C*  11-C*  12-B*  13-A
14-B  15-D  16-A  17-D  18-C  19-A  20-B  21-A*  22-B  23-D*  24-C*  25-A
26-B  27-D*  28-B  29-C*  30-D
```

* = OPTIONAL (Q1, 5, 10, 11, 12, 21, 23, 24, 27, 29). Regular: 1 pt if right, 0 if wrong. OPTIONAL: +2 right, −1 wrong, 0 blank. Part 1 = regular right + 2 × optional right − optional wrong (max 40). Part 2 and Part 3 rubrics are on the last pages.

part 1 Multiple Choice

40 pts · 1 pt each, OPTIONAL items +2 / −1 / blank 0

USE FOR QUESTIONS 1–3

```
class Gadget {
    string name;
    int power;
public:
    Gadget() : name("None"), power(0) { cout << "D "; }
    Gadget(string n) : name(n), power(10) { cout << "O1 "; }
    Gadget(string n, int p) : name(n), power(p) { cout << "O2 "; }
    Gadget(const Gadget& g) : name(g.name), power(g.power) { cout << "C "; }
    int getPower() const { return power; }
};

int main() {
    Gadget a;                  // (1)
    Gadget b("Laser");         // (2)
    Gadget c("Drill", 50);     // (3)
    Gadget d(c);               // (4)
    Gadget e = b;              // (5)
    Gadget f;                  // (6)
    f = c;                     // (7)
    cout << "| " << e.getPower() << endl;
}
```

1. OPTIONAL · +2 right · -1 wrong · 0 blank What does the whole program print?

A `D O1 O2 C D D | 10`
B `D O1 O2 C C D C | 10`
C `D O1 O2 C C D | 10`
D `D O1 O2 C C D | 0`

C · Lines 1–6 each build a new object (D O1 O2 C C D). Line 7 is assignment, not construction, so it prints nothing. `e` copied `b`, whose power is 10.

2. Line (5) `Gadget e = b;` is built by which constructor?

A the assignment operator
B the default constructor
C `Gadget(string n)`
D the copy constructor

D · `e` doesn't exist yet, so the `=` is copy-initialization: it calls the copy constructor (W04 Q7).

3. Line (7) `f = c;` does what?

A Calls the copy constructor and prints `C`
B Uses the assignment operator; no constructor runs, nothing prints
C Calls the default constructor and prints `D`
D Compile error: no `operator=` was written

B · `f` already exists, so this is assignment. The compiler-provided `operator=` copies the members silently.

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

4. Which data member MUST be given its value in a constructor's initializer list?

- **A** `const int id;`
- **B** `string name;`
- **C** `int count;`
- **D** `double* data;`

A · A `const` member must have its value before the constructor body runs. Assigning it in the body is a compile error (W04 Q12).

USE FOR QUESTION 5

```
class Pair {
    int a;
    int b;
public:
    Pair(int x) : b(x), a(b + 1) {}
    void print() const { cout << a << " " << b << endl; }
};
// main():  Pair p(5);  p.print();
```

5. `OPTIONAL · +2 right · -1 wrong · 0 blank` Which statement about `Pair p(5);` is true?

- **A** It prints `6 5`, because the list initializes `b` first
- **B** Compile error: the initializer list is in the wrong order
- **C** It prints `5 6`
- **D** `a` holds garbage: members are initialized in declaration order, so `a(b + 1)` runs before `b` is set

D · Initialization follows the order the members are declared in the class (`a`, then `b`), not the order in the list. It compiles, with a warning.

USE FOR QUESTION 6

```
class Buffer {
    int* data;
public:
    Buffer(int n) : data(new int[n]) {}
    ~Buffer() { delete[] data; }
    // no copy constructor written
};
// main():  Buffer b1(10);  Buffer b2(b1);
```

6. What goes wrong with `Buffer b2(b1);`?

- **A** Nothing: `b2` gets its own copy of the array
- **B** Compile error: no copy constructor exists
- **C** Shallow copy: both objects point at the same array, so it is deleted twice when they are destroyed
- **D** `b2.data` is set to `nullptr`

C · The default copy constructor copies the address, not the memory. Two destructors then free the same block (double free).

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

USE FOR QUESTION 7

```
class X {
public:
    X()  { cout << "X "; }
    ~X() { cout << "~X "; }
};
class Y : public X {
public:
    Y()  { cout << "Y "; }
    ~Y() { cout << "~Y "; }
};
class Z : public Y {
public:
    Z()  { cout << "Z "; }
    ~Z() { cout << "~Z "; }
};

int main() {
    cout << "start ";
    {
        Z z;
        cout << "| ";
    }
    cout << "end" << endl;
}
```

7. What does the program print?

A `start X Y Z | ~Z ~Y ~X end`

B `start Z Y X | ~Z ~Y ~X end`

C `start X Y Z | ~X ~Y ~Z end`

D `start X Y Z ~Z ~Y ~X | end`

A · Constructors run base-first (X Y Z). Destructors run in reverse (~Z ~Y ~X) when `z` leaves scope at the inner `}`, after the `|`.

USE FOR QUESTION 8

```
class Pet {
protected:
    string name;
public:
    Pet(string n) : name(n) {}
};
class Dog : public Pet {
    int tricks;
public:
    Dog(int t) : tricks(t) {}
};
// main():  Dog d(3);
```

8. What happens?

A It compiles; `name` is an empty string

B Compile error: `Pet` has no default constructor and `Dog` never chains to `Pet(string)`

C Compile error: `tricks` must be `protected`

D It compiles but crashes at run time

B · With no base call in the initializer list, the compiler tries `Pet()`, which doesn't exist (W05 Case B).

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

USE FOR QUESTIONS 9–11

```
class Pet {
    int age;
protected:
    string name;
public:
    Pet(string n, int a) : age(a), name(n) {}
    int getAge() const { return age; }
};

class Cat : public Pet {
public:
    Cat(string n, int a) : Pet(n, a) {}
    void info() {
        cout << name;          // (A)
        cout << age;           // (B)
        cout << getAge();      // (C)
    }
};

int main() {
    Cat c("Tom", 3);
    cout << c.name;            // (D)
    cout << c.getAge();        // (E)
}
```

9. Which lines do NOT compile?

A (A) and (D)

B (B) only

C (D) only

D (B) and (D)

D · (B): `age` is private to `Pet`. (D): `protected` opens access to derived classes only, not to `main()`.

10. `OPTIONAL · +2 right · -1 wrong · 0 blank` Why does line (B) fail?

A `age` is not inherited by `Cat`

B It needs to be written `this→age`

C `age` is private: every `Cat` contains it, but only `Pet`'s own code may touch it

D `Cat` must be a `friend` of `Pet`

C · Private members ARE inherited; they just aren't accessible. "Not inherited" is the classic wrong reason.

11. `OPTIONAL · +2 right · -1 wrong · 0 blank` The first line of `Cat` is changed to `class Cat : Pet` (no keyword). Which line ALSO fails now?

A (A)

B (C)

C (E)

D None: no keyword means public inheritance

C · No keyword on a `class` means private inheritance, so `getAge()` becomes private inside `Cat` and `main()` can't call it. (A) and (C) are inside `Cat` and still compile.

12. `OPTIONAL · +2 right · -1 wrong · 0 blank` With `class D : protected B`, a member that is `public` in `B` becomes ____ in `D`.

A public

B protected

C private

D not accessible

B · Protected inheritance caps everything at protected (lecture: Inheritance Access Levels table).

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

13. Inside an override `Cat::speak()`, which line runs `Pet`'s version first?

- **A** `Pet::speak();`
- **B** `super.speak();`
- **C** `this->speak();`
- **D** `speak();`

A · Scope resolution names the base version. `this->speak()` and `speak()` call `Cat::speak()` again, forever.

14. `Cat::eat()` and `Cat::eat(string food)` declared in the same class are:

- **A** overridden
- **B** overloaded
- **C** neither
- **D** a compile error

B · Same name, different parameter lists, same class: overloading. Overriding needs the same signature in a derived class.

15. Which of these should NOT be modeled with inheritance?

- **A** `Warrior : Character`
- **B** `Manager : Employee`
- **C** `Circle : Shape`
- **D** `Line : Point`

D · A Line is not a kind of Point; it HAS two Points. That's composition, not inheritance (lecture 04, is-a vs has-a).

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

USE FOR QUESTIONS 16–19

```
class Instrument {
public:
    virtual ~Instrument() {}
    void play()         { cout << "I-play "; }      // NOT virtual
    virtual void tune() { cout << "I-tune "; }
};
class Guitar : public Instrument {
public:
    void play()           { cout << "G-play "; }
    void tune() override  { cout << "G-tune "; }
    void strum()          { cout << "strum ";  }
};

int main() {
    Guitar g;
    Instrument* ip = &g;      // base POINTER to a Guitar
    Instrument& ir = g;       // base REFERENCE to a Guitar
    ip->play();     // (a)
    ip->tune();     // (b)
    ir.tune();      // (c)
    g.play();       // (d)
}
```

16. What does line (a) `ip->play();` print?

- **A** `I-play`
- **B** `G-play`
- **C** `I-play G-play`
- **D** Nothing: compile error

A · `play()` is not virtual, so the call is bound at compile time to the pointer's type, `Instrument`.

17. What do lines (b) and (c) print?

- **A** `I-tune I-tune`
- **B** `G-tune I-tune`
- **C** `I-tune G-tune`
- **D** `G-tune G-tune`

D · `tune()` is virtual, so a base pointer AND a base reference both run the actual object's version.

18. The line `ip->strum();` is added to `main()`. What happens?

- **A** It prints `strum`
- **B** It compiles and prints nothing
- **C** Compile error: `Instrument` has no member `strum`
- **D** It crashes at run time

C · The pointer's type decides what you may call; the object's type decides which version runs (W06 Case D).

19. Which call has the version that runs chosen at RUN time?

- **A** `ip->tune();`
- **B** `ip->play();`
- **C** `print(5);` where `print(int)` and `print(string)` exist
- **D** `maxOf(3, 7);` where `maxOf` is a template

A · Only a virtual function called through a base pointer/reference is dynamically bound. The others are all decided while compiling.

20. Function overloading, operator overloading, and templates are all examples of:

- **A** run-time polymorphism
- **B** compile-time (static) polymorphism
- **C** dynamic binding
- **D** object slicing

B · The compiler picks the version from the argument types; no vtable involved (lecture: Static Polymorphism).

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

USE FOR QUESTIONS 21–23 · TREAT EACH NUMBERED LINE AS THE ONLY FUNCTION IN DRAGON

```
class Monster {
public:
    virtual ~Monster() {}
    virtual void roar() const;
    virtual void move(int steps);
    void sleep();                                  // NOT virtual
};

class Dragon : public Monster {
public:
    void roar() const override;                    // (1)
    void roar() override;                          // (2)
    void move(double steps) override;              // (3)
    void sleep() override;                         // (4)
    void Move(int steps) override;                 // (5)
    void move(int steps);                          // (6)  no 'override'
};
```

21. OPTIONAL · +2 right · -1 wrong · 0 blank Which two lines compile?

A (1) and (6)

B (1) and (2)

C (2) and (6)

D (1) only

A · (1) is an exact match. (6) is also an exact match; `virtual` is inherited and `override` is optional.

22. Why is line (4) a compile error?

A It's missing `const`

B `sleep()` isn't virtual in `Monster`

C The name doesn't match

D The parameter types don't match

B · Only virtual functions can be overridden. (2) is missing const, (3) has the wrong parameter type, (5) is misspelled.

23. OPTIONAL · +2 right · -1 wrong · 0 blank `override` is deleted from line (2), so it compiles. Given `Monster* m = new Dragon();`, what does `m→roar();` run?

A `Dragon::roar()`

B Both, base version first

C Nothing: it crashes

D `Monster::roar()`, because `Dragon`'s version hides it instead of overriding it

D · Without `const` it's a different signature, so it never fills the `roar` slot in the vtable. No error, wrong output: the bug `override` prevents.

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

USE FOR QUESTIONS 24–25

```
class Animal {
public:
    virtual ~Animal() {}
    virtual void speak() { cout << "..." << endl; }
};
class Dog : public Animal {
public:
    void speak() override { cout << "Woof" << endl; }
};

void callVal(Animal a)  { a.speak(); }
void callRef(Animal& a) { a.speak(); }
void callPtr(Animal* a) { a->speak(); }

int main() {
    Dog d;
    callVal(d);                 // (a)
    callRef(d);                 // (b)
    callPtr(&d);                // (c)
    vector<Animal> zoo = {d};
    zoo[0].speak();             // (d)
}
```

24. `OPTIONAL · +2 right · -1 wrong · 0 blank` What do lines (a) and (d) print?

A `Woof` and `Woof`

B `...` and `Woof`

C `...` and `...`

D `Woof` and `...`

C · Both copy the `Dog` into a plain `Animal` (by-value parameter, `vector<Animal>`). Lines (b) and (c) print Woof.

25. What is this problem called, and what fixes `callVal`?

A Object slicing; change the parameter to `Animal& a`

B Shadowing; add `virtual` to `Dog::speak()`

C Hiding; add `override` to `Animal::speak()`

D Object slicing; add `virtual` to `Dog::speak()`

A · Pass by reference (or pointer) so no copy is made and `virtual` has a real `Dog` to find.

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

USE FOR QUESTIONS 26–27

```
class Engine {
public:
    ~Engine() { cout << "~Engine "; }           // not virtual
};
class Turbo : public Engine {
    double* boost;
public:
    Turbo() : boost(new double[50]) {}
    ~Turbo() { cout << "~Turbo "; delete[] boost; }
};

int main() {
    Engine* e = new Turbo();
    delete e;
}
```

26. As written, what does the program print?

A `~Turbo ~Engine`

B `~Engine`

C `~Engine ~Turbo`

D Nothing

B · Deleting through a base pointer with a non-virtual destructor runs only `~Engine`. The `boost` array leaks.

27. `OPTIONAL · +2 right · -1 wrong · 0 blank` Which one-word fix makes it clean up correctly, and what does it print after?

A `virtual` on `~Turbo()`; prints `~Turbo ~Engine`

B `delete[] e;` in main; prints `~Turbo ~Engine`

C `virtual` on `~Engine()`; prints `~Engine ~Turbo`

D `virtual` on `~Engine()`; prints `~Turbo ~Engine`

D · The base destructor must be virtual. Then the derived destructor runs first, base last. Rule: any virtual function → virtual destructor.

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

USE FOR QUESTIONS 28–30

```
class Shape {
protected:
    string label;
public:
    Shape(string l) : label(l) {}
    virtual ~Shape() {}
    virtual double perimeter() const = 0;       // pure virtual
    string getLabel() const { return label; }
};
class Square : public Shape {
    double side;
public:
    Square(double s) : Shape("square"), side(s) {}
    double perimeter() const override { return 4 * side; }
};
class Blob : public Shape {
public:
    Blob() : Shape("blob") {}
    double perimeter() { return 0; }
};
```

28. Which line does NOT compile?

- A `Shape* p = new Square(2);`
- B `Shape s("thing");`
- C `vector<Shape*> v;`
- D `Square sq(3); Shape& r = sq;`

B · You can't create an object of an abstract class. Pointers and references to one are fine.

29. `OPTIONAL · +2 right · -1 wrong · 0 blank` What happens with `Blob b;`?

- A It compiles; `perimeter()` returns 0
- B Compile error: `Shape` needs a default constructor
- C Compile error: `Blob::perimeter()` is missing `const`, so the pure virtual is never overridden and `Blob` is still abstract
- D It compiles but crashes

C · A missing `const` makes a new function instead of an override. Writing `override` would have pointed right at it (W06 Case B).

30. Which statement about vtables is TRUE?

- A Every object holds its own copy of the vtable
- B Non-virtual calls are looked up through the vtable
- C `main()` sets each object's vptr
- D There is one vtable per class; each object stores a hidden vptr to it

D · One shared table per class. The object's constructor sets its vptr. Non-virtual calls are resolved at compile time.

Test 2 — Constructors, Inheritance & Polymorphism

60-minute exam · 60 points — ANSWER KEY

answers Part 1 Answer Sheet

circle one letter per question

★	1.	A	B	**C**	D	★	11.	A	B	**C**	D	★	21.	**A**	B	C	D
	2.	A	B	C	**D**	★	12.	A	**B**	C	D		22.	A	**B**	C	D
	3.	A	**B**	C	D		13.	**A**	B	C	D	★	23.	A	B	C	**D**
	4.	**A**	B	C	D		14.	A	**B**	C	D	★	24.	A	B	**C**	D
★	5.	A	B	C	**D**		15.	A	B	C	**D**		25.	**A**	B	C	D
	6.	A	B	**C**	D		16.	**A**	B	C	D		26.	A	**B**	C	D
	7.	**A**	B	C	D		17.	A	B	C	**D**	★	27.	A	B	C	**D**
	8.	A	**B**	C	D		18.	A	B	**C**	D		28.	A	**B**	C	D
	9.	A	B	C	**D**		19.	**A**	B	C	D	★	29.	A	B	**C**	D
★	10.	A	B	**C**	D		20.	A	**B**	C	D		30.	A	B	C	**D**

★ = OPTIONAL: leave blank for 0, +2 if right, −1 if wrong.

Regular right ____ × 1 + ★ right ____ × 2 − ★ wrong ____ = Part 1 score _____ / 40